



Studio-Pogo

2 rue du Curé Marion

Immersion en Agence

5 Mai, 2026



Sommaires

Sommaires.....	1
1. Onboarding et Lancement du Projet.....	2
2. Conception et Directives Techniques.....	4
3. Initialisation et Base de Données.....	5
1. Laragon.....	5
2. Conception de la BDD.....	8
3. Création de l'architecture et initialisation de Composer.....	9
1. Sécurisation avec les variables d'environnement (.env).....	12
2. Mise en place du Routeur.....	13
3. Création de la classe Database.....	17
4. Exemple d'utilisation attendu dans tes Modèles :.....	20
4. Développement V1 : Architecture MVC.....	21
4.1. Création du Contrôleur.....	23
4.2. Intégration de l'interface : Création de la Vue.....	23
4.3. Gestion des données : Création du Modèle.....	24
4.4. Finalisation du cycle MVC : Transmission des données.....	25
1. Gestion des erreurs.....	26
2. Standards de l'Agence : Conventions de nommage et Méthodologie BEM.....	28
4.5. Création et Importation de la Feuille de Style BEM.....	30
1. Importation dans la Vue.....	31
5. Phase de Développement Autonome : Authentification, CRUD et Sécurité.....	32
5.1. Autonomie et Résolution de Problèmes.....	32
5.2. Authentification et Sessions Natives.....	33
5.3. Finalisation du CRUD Utilisateurs.....	33
5.4. Sécurité et Contrôle des Rôles.....	33
5.5. Rappel des Standards.....	34
5.5. Livrable : Documentation Technique.....	34

1. Onboarding et Lancement du Projet

Bienvenue à l'agence, Timéo. Au nom de toute l'équipe, je te souhaite une **excellente intégration** parmi nous. Prends le temps de t'installer à ton poste de travail, qui est désormais entièrement configuré pour tes besoins. Je suis **Yulian**, développeur en alternance, et je serai ton collègue direct tout au long de ton immersion. À mes côtés (en face) se trouve **Quentin** (qui est en congé), notre développeur expérimenté et, surtout, ton tuteur attitré. C'est vers lui que tu devras te tourner pour valider tes choix techniques ou surmonter les blocages complexes. Ensemble, nous formons le pôle développement, le moteur technique de l'entreprise.

Pour que tu puisses prendre tes marques rapidement et travailler sereinement, il est essentiel que tu comprennes l'écosystème dans lequel tu viens de mettre les pieds. Notre structure est à taille humaine, ce qui exige une grande rigueur méthodologique et une communication parfaitement fluide. À la tête de l'agence, nous avons **Valéry** et **Gérald**, nos deux gérants. Ils portent la double casquette de directeurs de création et de stratèges ; ce sont eux qui définissent la vision globale, négocient les contrats et assurent le lien direct avec nos commanditaires. Au sein du pôle créatif, tu seras amené à interagir avec **Mathilde** (le poste à gauche), notre designeuse. Sa mission est cruciale : elle traduit les concepts abstraits des gérants en parcours utilisateurs fluides et en interfaces (UI/UX) esthétiques, ergonomiques et prêtes à être intégrées.

Le flux de production de l'agence obéit à une mécanique stricte dont tu vas très vite saisir les rouages. Le cycle de vie d'un projet se déroule ainsi : le besoin est qualifié par **Valéry** ou **Gérald**, **Mathilde** prend ensuite le relais pour concevoir les maquettes, puis le dossier est transféré à la technique. À cette étape, **Quentin** étudie la faisabilité et impose l'architecture ainsi que la stack technologique à employer. C'est seulement après ce processus que nous entrons en scène pour la phase de code pur. Il est également fondamental que tu gardes à l'esprit une réalité de notre métier : un développement n'est **jamais un long fleuve tranquille**. Tu seras très régulièrement confronté à des retours et des demandes de pivotement de la part des gérants ou des clients pendant la phase de production. Ces itérations exigent de la flexibilité, un code maintenable et une grande capacité d'adaptation. Enfin, la **livraison ne marque pas la fin de ta mission**, puisqu'aucun projet n'est validé sans la rédaction d'une **documentation** exhaustive.

D'ailleurs, ton immersion débute sur les chapeaux de roues. **Gérald** sort tout juste de réunion et vient de nous déposer le cahier des charges de ton premier projet. Il s'agit d'une commande interne stratégique, un socle technique destiné à être déployé sur nos futurs produits. L'agence nécessite un système back-office de gestion d'utilisateurs sur mesure, sécurisé et performant.

Le brief de **Gérald** est précis : tu as la responsabilité de concevoir de A à Z une architecture permettant l'administration complète des comptes (le fameux système **CRUD** pour créer, lire, modifier et supprimer des entités). La complexité et le cœur de ce projet résident dans la gestion granulaire des **rôles**. Ton code devra être capable **d'identifier, d'authentifier** et de **restreindre** les permissions selon les profils connectés. L'application doit impérativement distinguer un rôle "**Administrateur**", possédant les pleins pouvoirs sur la plateforme pour interagir avec l'ensemble des données, d'un rôle "**Utilisateur**", dont l'environnement sera verrouillé et restreint. Nous attendons de toi une base de données optimisée, une sécurité infaillible et une intégration fidèle aux maquettes que **Mathilde** finalise en ce moment même. C'est un défi technique formateur qui te plongera directement dans nos standards de qualité.

Nous avons hâte de voir tes premières lignes de code.

2. Conception et Directives Techniques

L'étape de conception est maintenant achevée. **Mathilde** vient de finaliser les **maquettes** de l'interface et te les transfère à l'instant. Lors de la présentation de la cible visuelle, elle insiste sur deux points fondamentaux : **Jujutsu Kaisen est un super animé** et la **fidélité absolue de l'intégration**. L'interface que tu vas développer doit être le reflet exact de son travail. Les espacements, la hiérarchie typographique, les codes couleurs et les comportements visuels ont été minutieusement pensés et validés par Valéry et Gérald. Tu te dois donc de respecter **strictement** ce design, au **pixel** près. Mathilde reste bien entendu à ta disposition si tu as besoin **d'exporter des assets spécifiques** ou de **clarifier** une interaction complexe prévue sur la maquette, mais aucune initiative altérant l'ergonomie ou l'interface validée ne sera tolérée sans une concertation préalable avec elle.

Lien Figma vers la maquette : [LIEN](#)

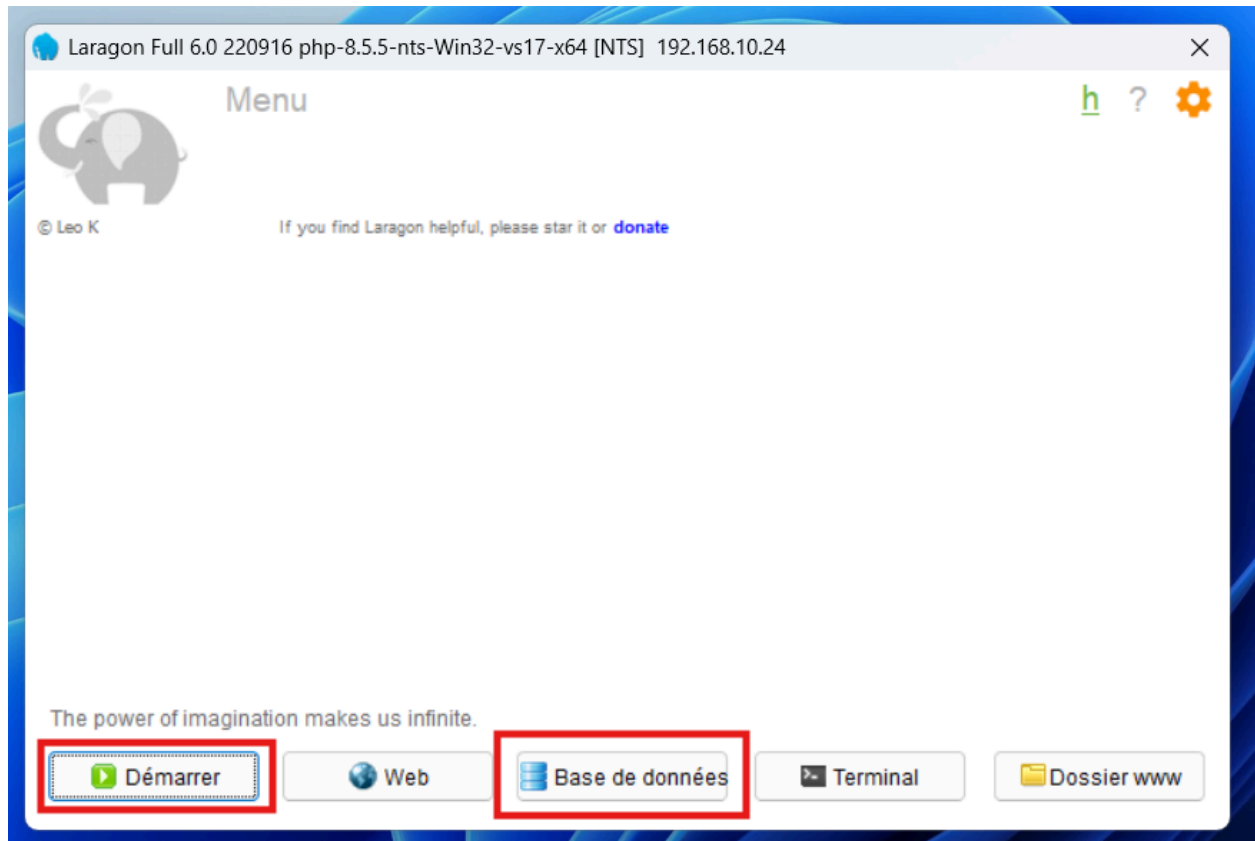
Une fois la cible visuelle assimilée, c'est au tour de Quentin d'intervenir pour fixer le cadre technique. En tant que responsable technique, il a défini une stack obligatoire que tu devras impérativement appliquer. Pour cette mission, nous écartons volontairement l'usage de frameworks **PHP** complexes afin d'évaluer et de consolider tes fondamentaux. Tu vas commencer par configurer ton environnement de développement local en utilisant **Laragon**. Le cœur du système devra être développé en **PHP Vanilla**, en appliquant rigoureusement les principes de la **Programmation Orientée Objet (POO)**. Pour la gestion des données, l'utilisation de l'extension **PDO** est exigée pour toutes tes requêtes, garantissant ainsi la sécurité de notre base. Quentin impose également la mise en place d'une architecture de dossiers claire, logique et modulaire, s'inspirant fortement du modèle MVC pour séparer distinctement la logique métier, la communication avec la base de données et les vues.

Côté front-end, l'exigence technique est tout aussi élevée. Ton code **HTML** devra être parfaitement sémantique. Pour l'intégration des maquettes de Mathilde, l'écriture du **CSS** devra obéir scrupuleusement à la méthodologie **BEM (Block Element Modifier)**. Cette approche est non négociable chez nous : elle garantit un code stylistique isolable, réutilisable et maintenable par n'importe quel autre développeur de l'équipe. Enfin, pour dynamiser les interfaces, gérer les validations de formulaires côté client ou effectuer des appels asynchrones, tu devras utiliser exclusivement du JavaScript natif (**Vanilla JS**). L'utilisation de bibliothèques externes est proscrite pour ce projet. Le périmètre technique est défini et les attentes sont claires, l'environnement est prêt à être initialisé.

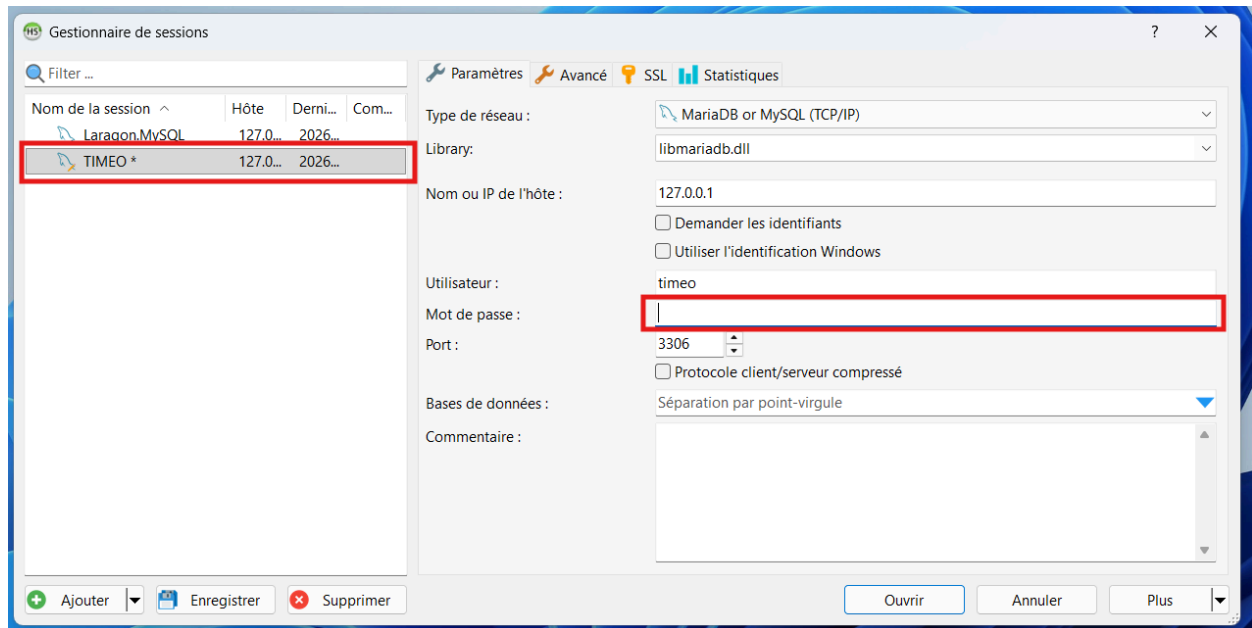
3. Initialisation et Base de Données

Commençons dès maintenant. Tout d'abord je vais te montrer à quel session HeidiSQL te connecter, puis t'aider à créer ton **arborescence (structure de projet)**.

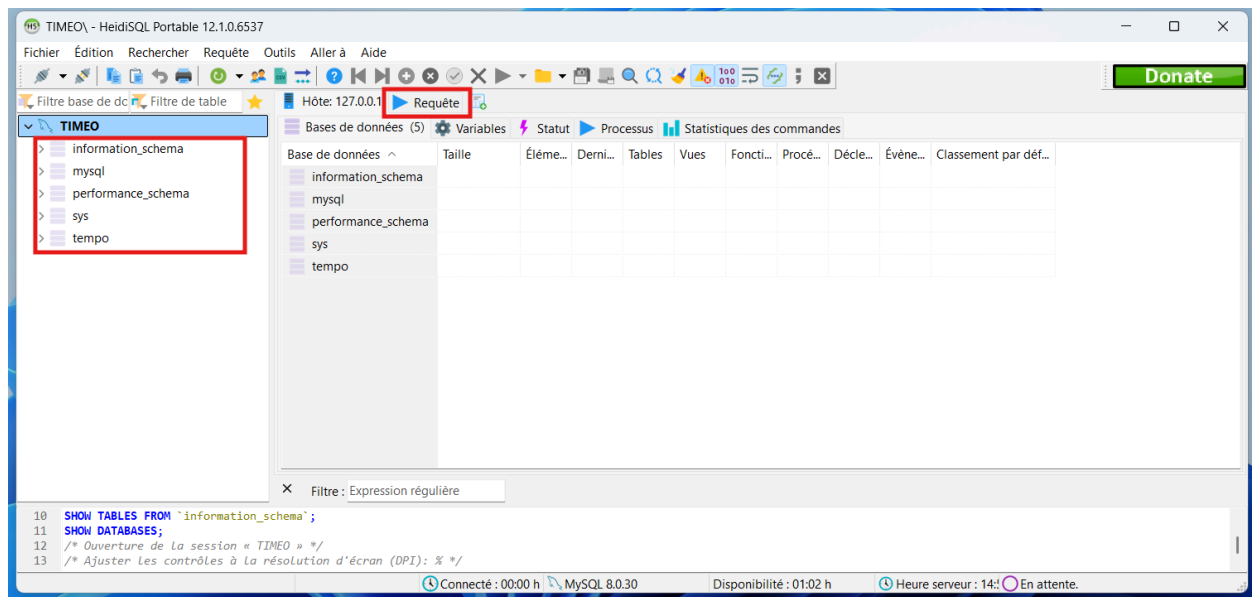
1. Laragon



Tu peux démarrer les services de Laragon. Il y en a deux : **Apache** (serveur web) et **MySQL** (serveur de base de données relationnelles : SGBDR). Commence donc par démarrer les services et rendons nous sur **HeidiSQL** (logiciel open-source installé dans **Laragon** pour gérer les SGBDR).

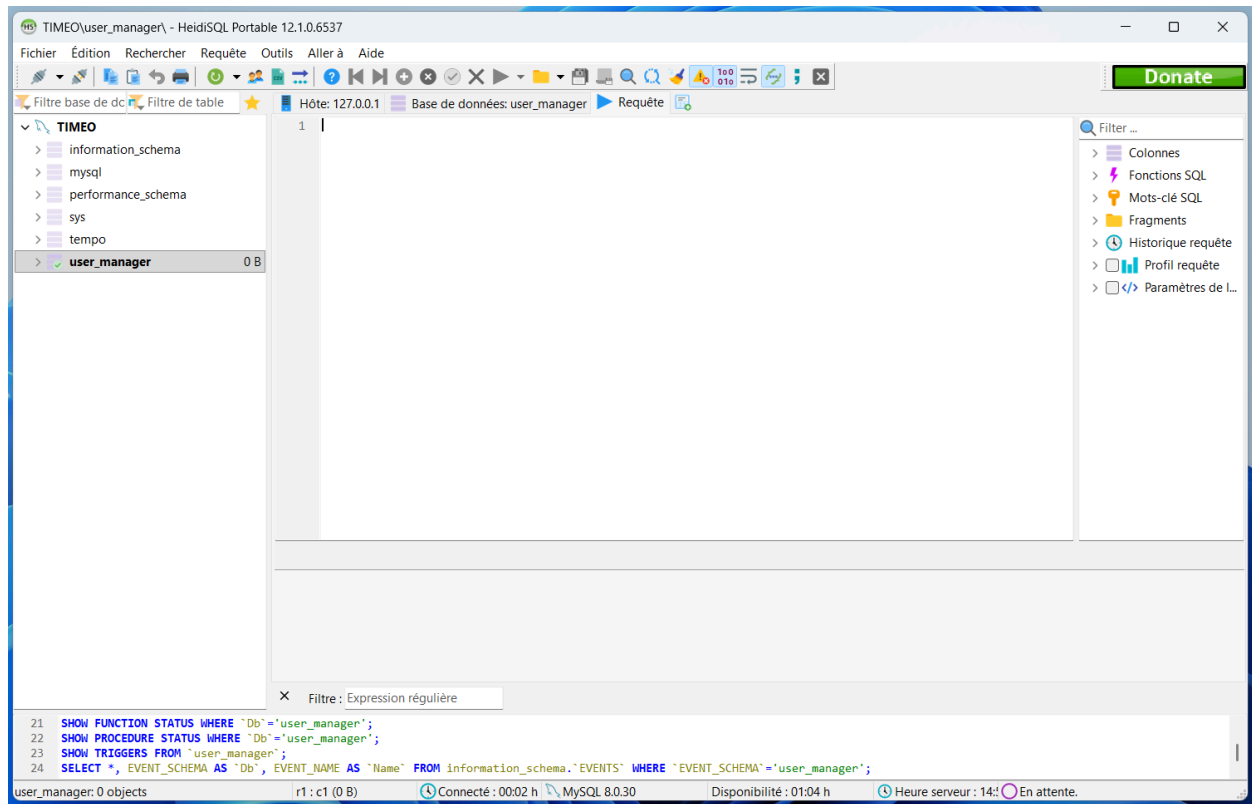


Sélectionne bien ta session, puis rentre ton mot de passe (celui de ton ordinateur, sur le post-it)



Je suis sûr que tu connais déjà l'interface de Laragon. Pour faire simple, sur la barre de gauche il y a la liste des bases de données de la session, et à droite le contenu. Tu peux entrer des requêtes dans l'onglet Requête.

Tu peux créer une base de données nommée “user_manager”. Voici l’interface souhaitée :



2. Conception de la BDD.

Il faut donc deux tables distinctes : **roles** et **users**. La table roles doit définir les niveaux d'accès avec un identifiant et un nom (ex: **'admin'**, **'user'**). La table users stockera les informations de chaque compte (**id**, **nom**, **email**, **mot de passe**). Ajoute une clé étrangère dans la table users pour la lier à l'identifiant de la table roles.

TIMEO\user_manager\ - HeidiSQL Portable 12.1.0.6537

Fichier Édition Rechercher Requête Outils Aller à Aide

Hôte: 127.0.0.1 Base de données: user_manager Requête

Donner

Filter base de données Filter de table

TIMEO

- information_schema
- mysql
- performance_schema
- sys
- tempo
- user_manager 48,0 KiB
 - roles 16,0 KiB
 - users 32,0 KiB

Nom	Lignes	Taille	Créé le	Mis à jour	Moteur	Commentaire	Type
roles	0	16,0 KiB	2026-05-05 15:10...		InnoDB		Table
users	0	32,0 KiB	2026-05-05 15:10...		InnoDB		Table

Filter: Expression régulière

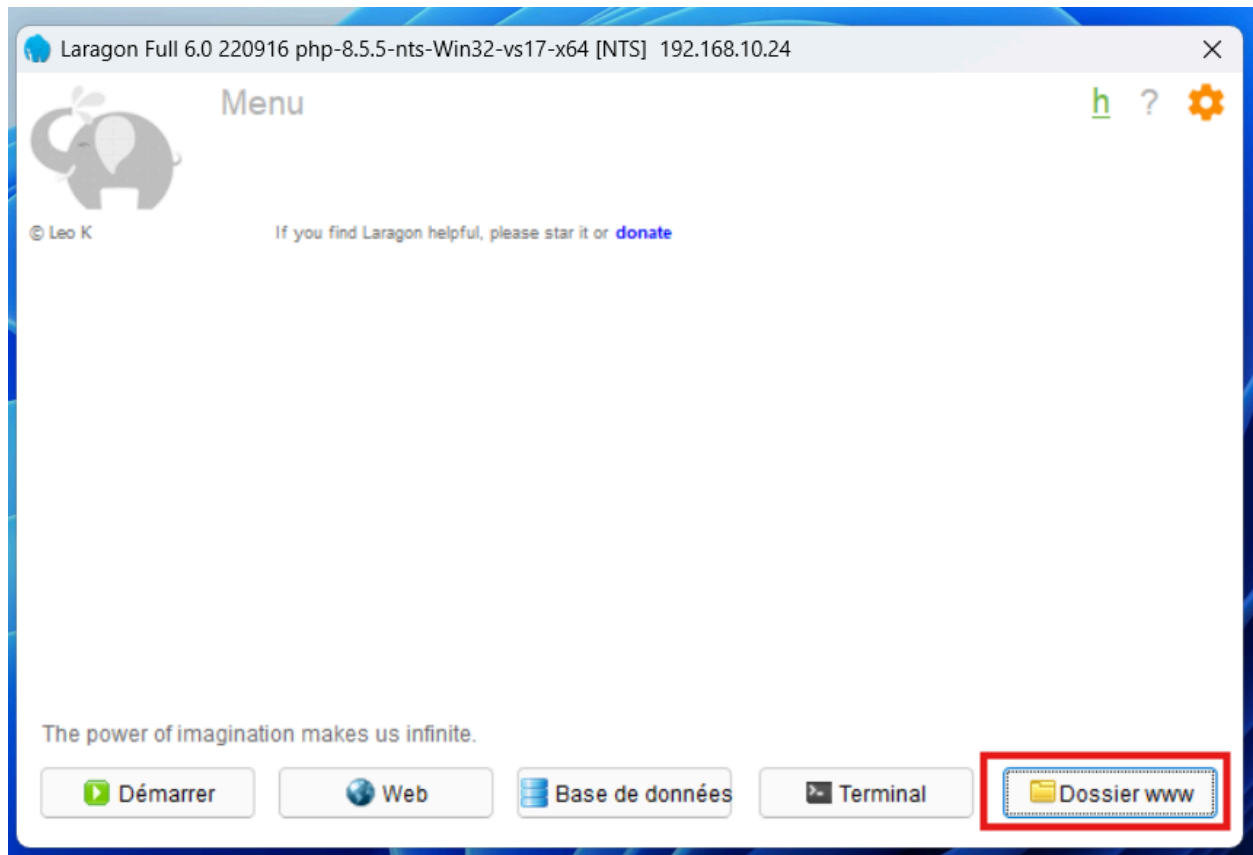
```

102 SELECT * FROM information_schema.KEY_COLUMN_USAGE WHERE TABLE_SCHEMA='user_manager' AND TABLE_NAME='users' AND REFERENCED_TABLE_NAME IS NOT NULL;
103 SHOW CREATE TABLE `user_manager`.`users`;
104 SELECT tc.CONSTRAINT_NAME, cc.CHECK_CLAUSE FROM `information_schema`.`CHECK_CONSTRAINTS` AS cc, `information_schema`.`TABLE_CONSTRAINTS` AS tc WHERE tc.CONSTRAINT_SCHEMA='user_'
105 SELECT * FROM `user_manager`.`users` LIMIT 1000;
  
```

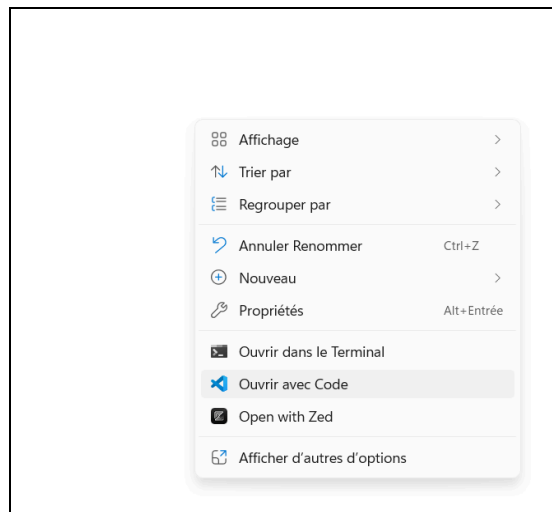
Connecté : 00:15 h MySQL 8.0.30 Disponibilité : 01:16 h Heure serveur : 15: En attente.

3. Création de l'architecture et initialisation de Composer

Commence par préparer le terrain dans ton répertoire Laragon. Depuis ce dernier, tu peux facilement accéder au répertoire :



Tu peux créer un dossier nommé par le nom de ton projet. Pour notre part c'est 'user_manager'. Puis ouvre VSCode sur ce répertoire.



J'imagine que tu dois déjà être familier avec l'interface de VSCode. Peux tu ouvrir le terminal intégré de VSCode et taper `"composer init"`.

```

PS C:\laragon\www\user_manager> composer init

Welcome to the Composer config generator

This command will guide you through creating your composer.json config.

Package name (<vendor>/<name>) [yulian/user_manager]:
Description []:
Author [Yulian <yulian@studio-pogo.fr>, n to skip]:
Minimum Stability []:
Package Type (e.g. library, project, metapackage, composer-plugin) []:
License []:

Define your dependencies.

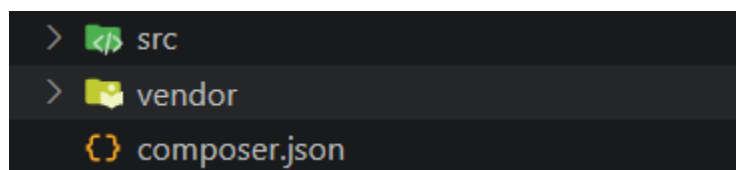
Would you like to define your dependencies (require) interactively [yes]?
Search for a package:
Would you like to define your dev dependencies (require-dev) interactively [yes]?
Search for a package:
Add PSR-4 autoload mapping? Maps namespace "Yulian\UserManager" to the entered relative path. [src/, n to skip]:

{
    "name": "yulian/user_manager",
    "autoload": {
        "psr-4": {
            "Yulian\\UserManager\\": "src/"
        }
    },
    "authors": [
        {
            "name": "Yulian",
            "email": "yulian@studio-pogo.fr"
        }
    ],
    "require": {}
}

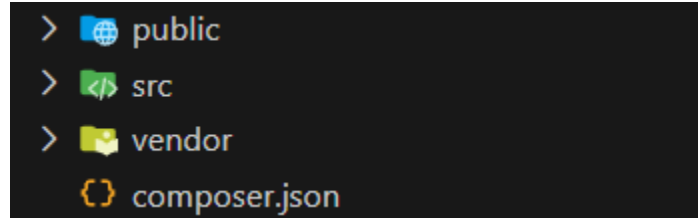
Do you confirm generation [yes]? █

```

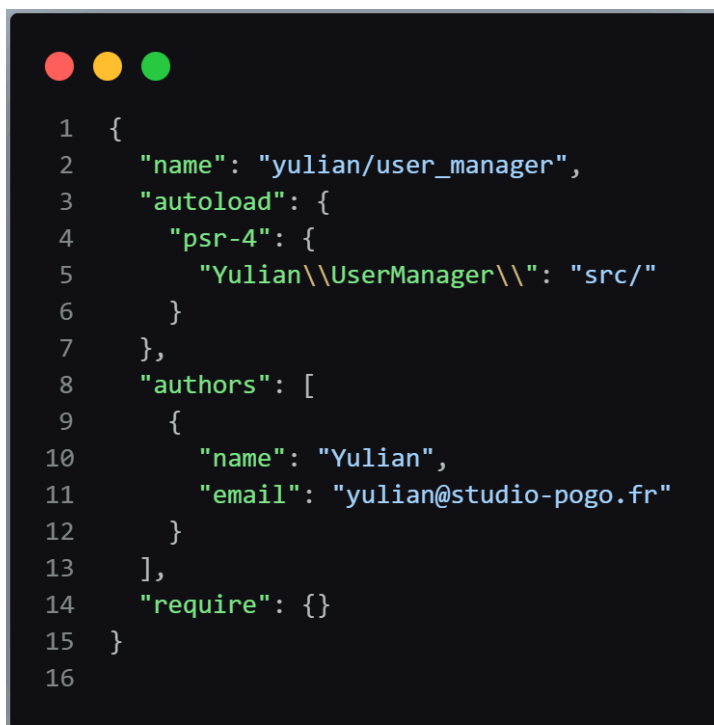
Pour ma part j'ai laissé les paramètres par défaut et tapant "Entrer" plusieurs fois. Ce sont des configuration de composer. Le plus important est le namespace car il te permettra d'importer des classes correctement. Pour moi c'est "Yulian\UserManager" mais toi ça peut être autre chose. Je peux donc confirmer. Cela me créer cette arborescence :



Par la même occasion tu peux créer un dossier **public/** qui sera la racine web contenant tes fichiers statiques (CSS, JS) et ton point d'entrée. Le dossier **src/** abritera toute ta logique métier (Modèles, Contrôleurs).



Par convention dans le monde professionnel, nous utilisons le namespace "App" alors que pour le moment le mien est "Yulian\UserManager" et toi encore autre chose. Nous allons donc modifier dans **composer.json** le namespace :



Dans "psr-4" (norme de import) je vais remplacer "Yulian\\UserManager\\" par "App".



Lance ensuite un `composer dump-autoload`. Grâce à cela, tu n'auras plus jamais besoin d'utiliser **require** ou **include** pour charger tes classes PHP ; Composer le fera automatiquement.

```
PS C:\laragon\www\user_manager> composer dump-autoload
Composer could not detect the root package (yulian/user_manager) version, defaulting to '1.0.0'. See https://getcomposer.org/root-version
Generating autoload files
Generated autoload files
```

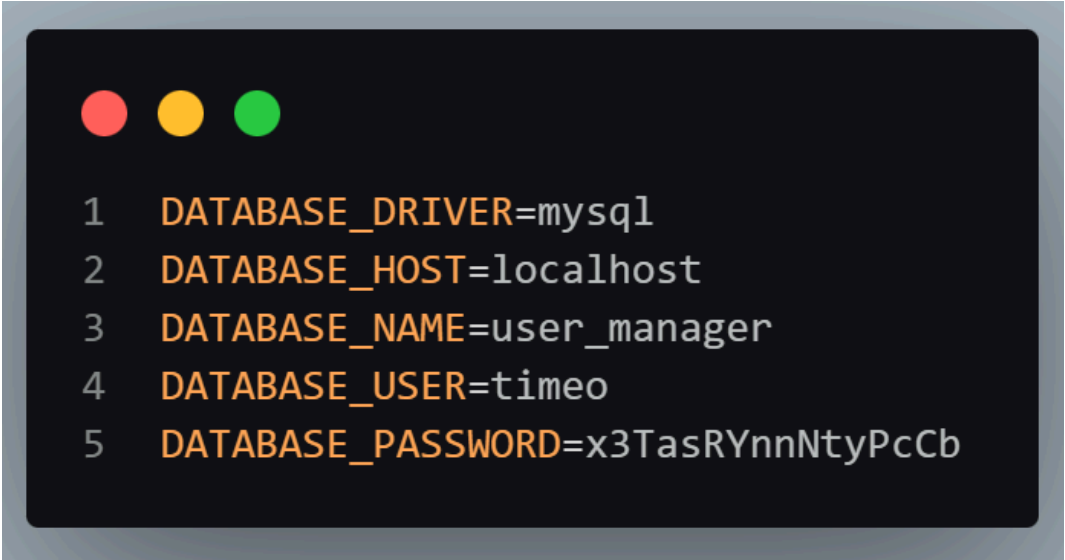
1. Sécurisation avec les variables d'environnement (.env)

La règle d'or chez nous : on ne stocke jamais d'identifiants en dur dans le code. Installe la librairie **Dotenv** via la commande `composer require vlucas/phpdotenv`.

Crée un fichier nommé `.env` à la racine de ton projet. Tu vas y déclarer les variables de connexion à ta base de données (**hôte, nom de la base, utilisateur, mot de passe**).

Immédiatement après, crée un fichier `.gitignore` et ajoutes-y le fichier `.env` ainsi que le dossier **vendor/**. C'est impératif pour éviter toute fuite de données si nous passions le projet sur un dépôt Git.

Voici le contenu du `.env` :



```
1 DATABASE_DRIVER=mysql
2 DATABASE_HOST=localhost
3 DATABASE_NAME=user_manager
4 DATABASE_USER=timeo
5 DATABASE_PASSWORD=x3TasRYnnNtyPcCb
```

2. Mise en place du Routeur

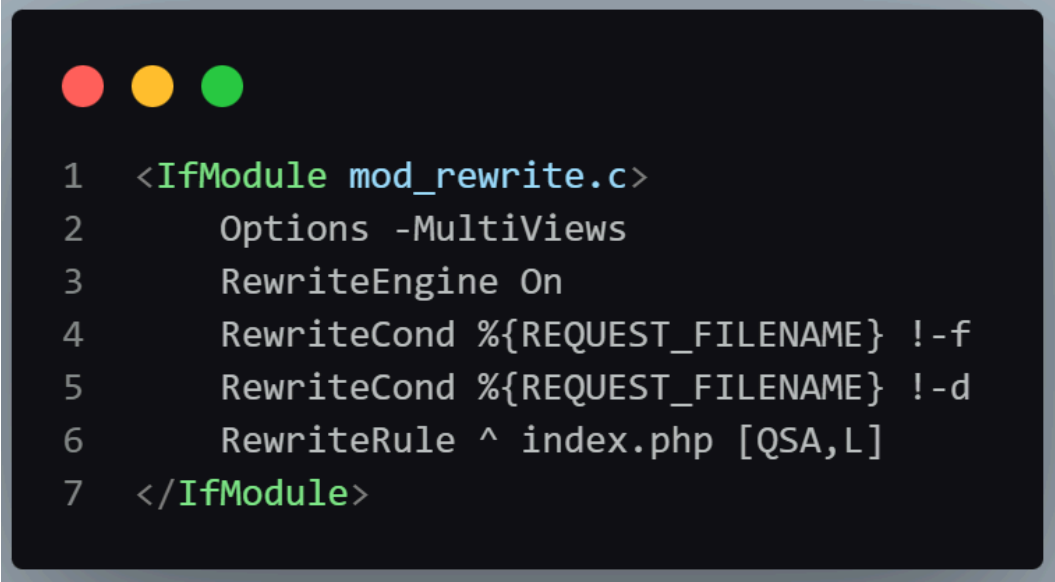
Pour le routage, Quentin te conseille **AltoRouter**, une librairie légère et parfaite pour du PHP Vanilla. Installe-la avec `composer require altorouter/altorouter`.

Ton application doit fonctionner avec un "Front Controller". Cela signifie que toutes les requêtes des utilisateurs doivent passer par un seul et unique fichier. Dans ton dossier **public/**, crée un fichier **index.php**. Tu devras également y placer un fichier **.htaccess** pour forcer **Apache** (via Laragon) à rediriger toutes les URLs vers cet **index.php**.

Dans cet **index.php**, tu vas **require** l'autoloader de Composer, initialiser Dotenv pour charger tes variables de configuration, puis instancier **AltoRouter**. C'est ici que tu vas "mapper" tes routes, par exemple indiquer que l'URL /login appelle la méthode d'affichage du formulaire dans ton contrôleur d'authentification.

Étape 1 : Configuration du serveur (Fichier .htaccess)

Le pattern "**Front Controller**" nécessite de rediriger tout le trafic web vers un seul fichier. Dans ton dossier **public/**, crée un fichier **.htaccess** et insères-y les directives suivantes pour le serveur Apache :

A screenshot of a code editor with a dark background and light-colored text. At the top left, there are three colored circles: red, yellow, and green. The code is as follows:

```
1 <IfModule mod_rewrite.c>
2     Options -MultiViews
3     RewriteEngine On
4     RewriteCond %{REQUEST_FILENAME} !-f
5     RewriteCond %{REQUEST_FILENAME} !-d
6     RewriteRule ^ index.php [QSA,L]
7 </IfModule>
```

Étape 2 : Chargement des dépendances et de l'environnement

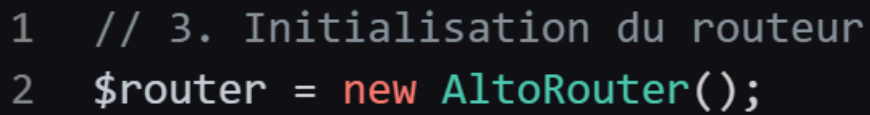
Dans le fichier **public/index.php**. Commence par charger l'**autoloader** de Composer. Ensuite, initialise **vlucas/phpdotenv** pour charger tes identifiants depuis le fichier **.env** situé à la racine du projet.



```
1  <?php
2
3  // 1. Autoloader de Composer
4  require_once __DIR__ . '/../vendor/autoload.php';
5
6  // 2. Chargement des variables environnement
7  $dotenv = Dotenv\Dotenv::createImmutable(__DIR__ . '/../');
8  $dotenv->load();
```

Étape 3 : Initialisation d'AltoRouter

Toujours dans **index.php**, instancie le routeur.



```
1  // 3. Initialisation du routeur
2  $router = new AltoRouter();
```

Étape 4 : Création des routes (Mapping)

Déclare les chemins de ton application avec la méthode `map()`. La convention est d'utiliser une chaîne de caractères contenant le nom du Contrôleur et la méthode à appeler, séparés par un #.

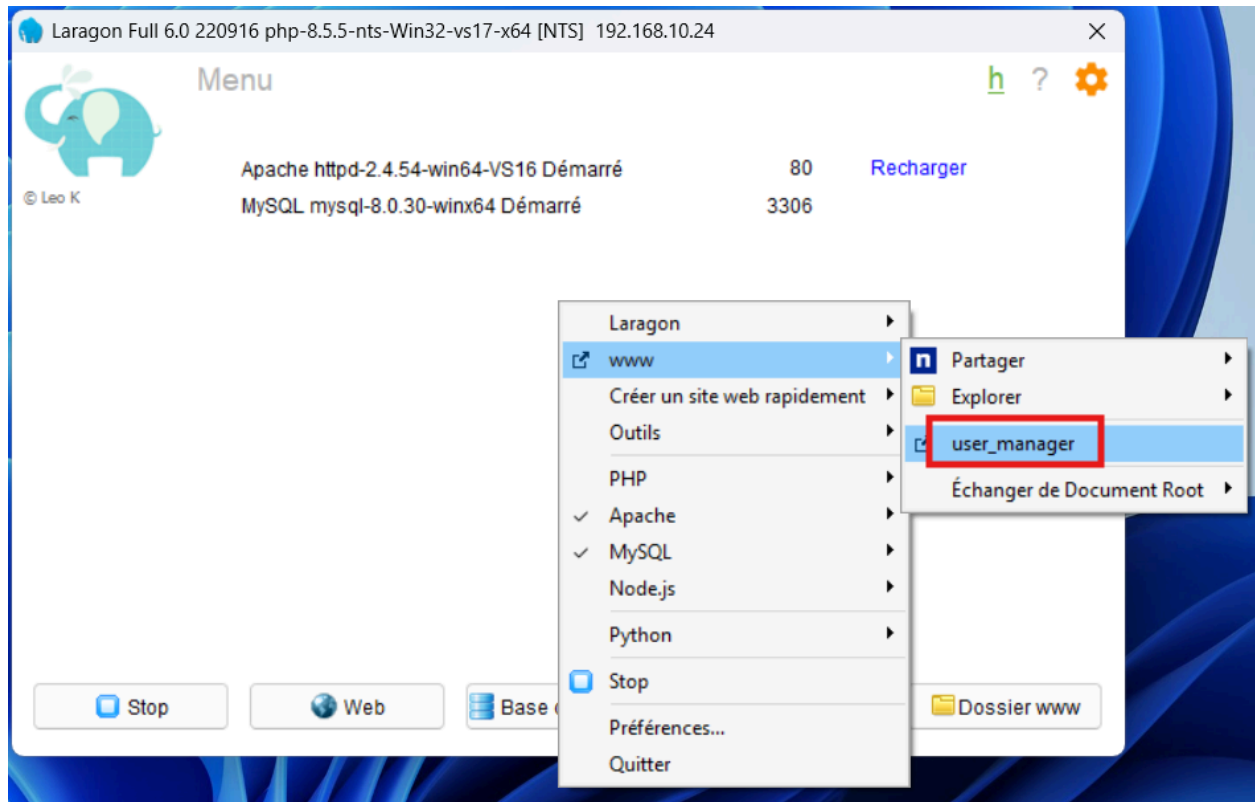
```
1 // 4. Declaration des routes (Methode, URL, Cible, Nom de la route)
2 $router->map('GET', '/', 'HomeController#index', 'home');
```

Étape 5 : Exécution de la route (Matching & Dispatching)

Demande au routeur de vérifier si l'URL saisie par l'utilisateur correspond à l'une des routes déclarées. Si oui, instancie dynamiquement le contrôleur et appelle la méthode. Si non, renvoie un statut HTTP 404.

```
1 // 5. Verification de URL
2 $match = $router->match();
3
4 if (is_array($match)) {
5     // Separation du contrôleur et de la methode (ex: AuthController et showLogin)
6     list($controller, $action) = explode('#', $match['target']);
7
8     // Ajout du namespace defini dans composer.json
9     $controllerName = "App\\Controllers\\" . $controller;
10
11     // Instanciation de la classe et appel de la methode avec ses parametres
12     $obj = new $controllerName();
13
14     if (is_callable([$obj, $action])) {
15         call_user_func_array([$obj, $action], $match['params']);
16     }
17 } else {
18     // Aucune route correspondante
19     echo 'Erreur 404 - Page introuvable';
20 }
```


Je t'invite dès à présent à ouvrir le projet Laragon :



Si jamais tu ne vois pas ton projet, redémarre Laragon. Tu devrais ouvrir dans le navigateur : ["http://user_manager.test/"](http://user_manager.test/). Tu devrais voir une erreur de ce type :

Fatal error: Uncaught Error: Class "App\Controllers\HomeController" not found in C:\laragon\www\user_manager\public\index.php:27 Stack trace: #0 {main} thrown in C:\laragon\www\user_manager\public\index.php on line 27

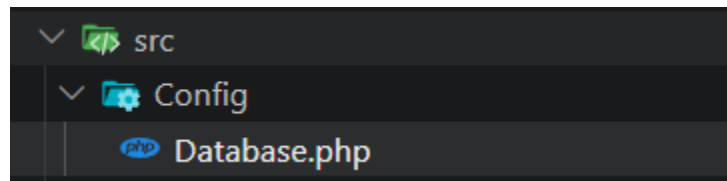
Si c'est le cas c'est totalement normal car nous n'avons pas créer de controller.

3. Création de la classe Database

Enfin, dans ton dossier **src/**, nous allons créer la classe Database. Quentin exige l'utilisation du design pattern "Singleton". Le concept est simple : tu vas créer une classe avec un constructeur privé et une méthode statique getInstance(). Cette méthode vérifiera si une connexion PDO (utilisant les variables de ton **.env**) existe déjà. Si c'est le cas, elle la retourne ; sinon, elle l'instancie. Cela garantit qu'une seule connexion à la base de données est ouverte par exécution de script, optimisant ainsi les performances du serveur.

Étape 1 : Structure des dossiers

Crée le dossier **Config** à l'intérieur du dossier **src/**. Puis crée le fichier **Database.php** dans ce nouveau dossier. L'arborescence est **src/Config/Database.php**.



Étape 2 : Namespace et propriétés

Ouvre **Database.php**. Déclare le **namespace App\Config** (qui correspond au standard PSR-4 défini dans composer).

Déclare deux propriétés privées : l'instance **statique** du Singleton et l'objet de **connexion PDO**.

```
1  <?php
2
3  namespace App\Config;
4
5  use PDO;
6  use PDOException;
7  use Exception;
8
9  class Database
10 {
11     private static ?Database $instance = null;
12     private PDO $connection;
13 }
```

Étape 3 : Constructeur privé

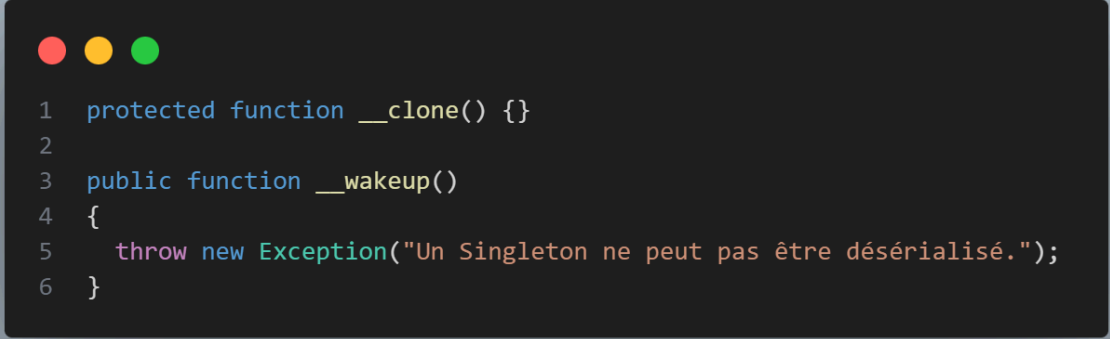
Le constructeur est privé pour bloquer l'instanciation classique via le mot-clé **new**. Récupère les identifiants via la superglobale **\$_ENV** (précédemment chargée par **phpdotenv** dans **index.php**).

Instancie PDO avec des options strictes de gestion d'erreurs et de sécurité.

```
1 private function __construct()
2 {
3     $driver = $_ENV['DATABASE_DRIVER'];
4     $host = $_ENV['DATABASE_HOST'];
5     $db = $_ENV['DATABASE_NAME'];
6     $user = $_ENV['DATABASE_USER'];
7     $pass = $_ENV['DATABASE_PASSWORD'];
8     $charset = 'utf8mb4';
9
10    $dsn = "$driver:host=$host;dbname=$db;charset=$charset";
11
12    $options = [
13        PDO::ATTR_ERRMODE           => PDO::ERRMODE_EXCEPTION,
14        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
15        PDO::ATTR_EMULATE_PREPARES  => false,
16    ];
17
18    try {
19        $this->connection = new PDO($dsn, $user, $pass, $options);
20    } catch (PDOException $e) {
21        die("Erreur de connexion : " . $e->getMessage());
22    }
23 }
```

Étape 4 : Sécurisation du Singleton

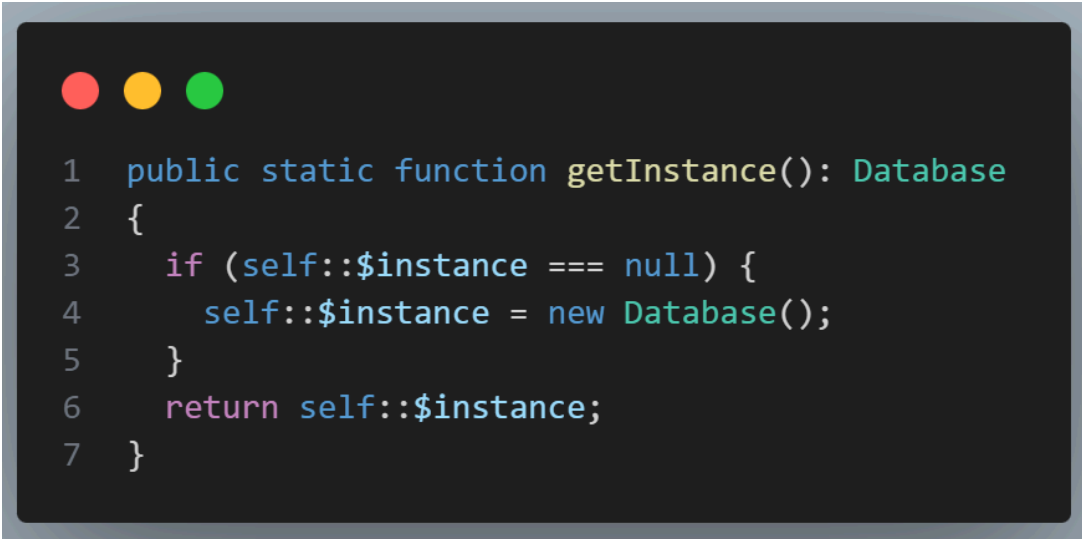
Surcharge les méthodes magiques de clonage et de désérialisation pour empêcher toute duplication accidentelle de l'instance.



```
1  protected function __clone() {}
2
3  public function __wakeup()
4  {
5      throw new Exception("Un Singleton ne peut pas être désérialisé.");
6  }
```

Étape 5 : Instanciation statique (getInstance)

C'est le cœur du design pattern. La méthode vérifie si l'instance existe déjà. Si ce n'est pas le cas, elle exécute le constructeur. Elle retourne toujours la même instance.



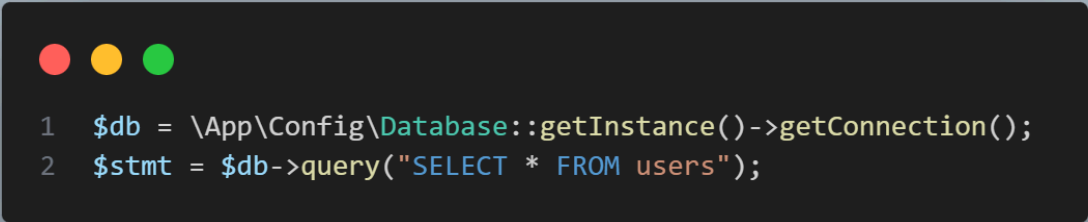
```
1  public static function getInstance(): Database
2  {
3      if (self::$instance === null) {
4          self::$instance = new Database();
5      }
6      return self::$instance;
7  }
```

Étape 6 : Accesseur de la connexion

Ajoute une méthode publique permettant aux autres classes du projet de récupérer l'objet PDO actif.

4. Exemple d'utilisation attendu dans tes Modèles :

Pour exécuter une requête SQL dans l'application, l'appel sera strictement formaté ainsi :

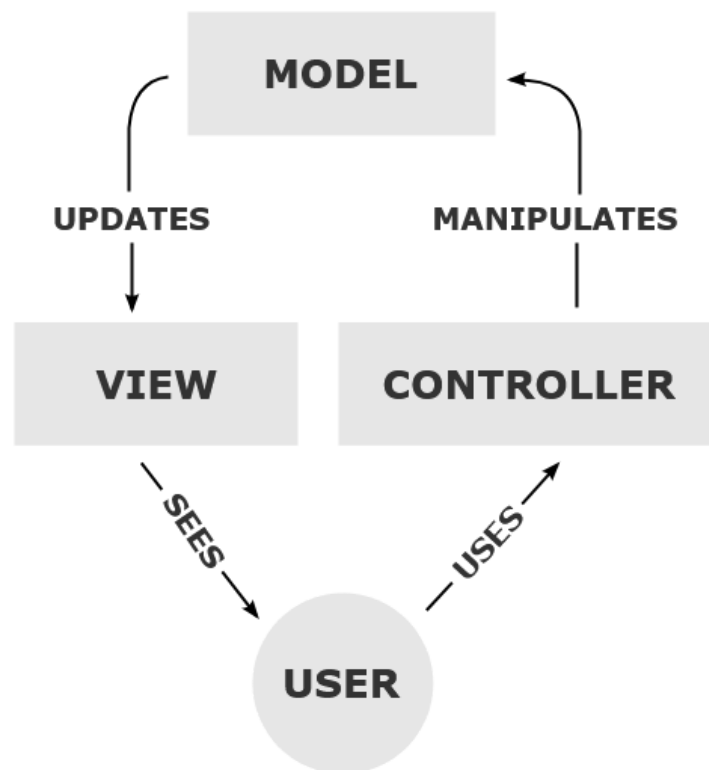


```
1 $db = \App\Config\Database::getInstance()->getConnection();  
2 $stmt = $db->query("SELECT * FROM users");
```

4. Développement V1 : Architecture MVC

Le projet repose sur le pattern d'architecture MVC (Modèle-Vue-Contrôleur). Ce standard de développement sépare les responsabilités du code en trois composants distincts pour garantir la maintenabilité et la lisibilité.

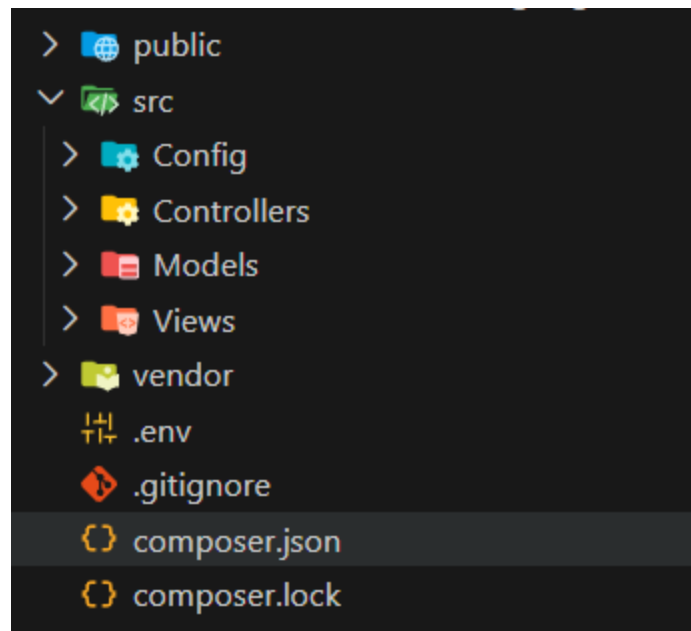
- **Modèle (Model)** : Gère la logique métier et la manipulation des données. Les classes Modèle communiquent directement avec la base de données via l'instance Singleton Database pour effectuer les opérations CRUD.
- **Vue (View)** : Représente l'interface utilisateur. Elle contient le code HTML et l'intégration CSS (méthodologie BEM). Elle ne contient aucune logique de traitement de données ; elle affiche uniquement les informations reçues.
- **Contrôleur (Controller)** : Agit comme l'intermédiaire de l'application. Il réceptionne la requête HTTP envoyée par le routeur (index.php), interroge le Modèle nécessaire pour récupérer ou modifier des données, puis charge et renvoie la Vue correspondante à l'utilisateur.



Mise en place de l'architecture

Crée la structure de dossiers suivante dans le répertoire **src/** :

- **src/Controllers/**
- **src/Models/**
- **src/Views/**



L'étape suivante consistera à créer le premier contrôleur (**HomeController**) dans le dossier Controllers et sa vue associée pour initier le premier cycle MVC complet.

4.1. Création du Contrôleur

L'erreur signalée précédemment provient de l'absence de la classe ciblée par le routeur. Crée le fichier **src/Controllers/HomeController.php**. Ce fichier réceptionne la requête initiale et charge l'interface.

```
1  <?php
2
3  namespace App\Controllers;
4
5  class HomeController
6  {
7      public function index(): void
8      {
9          require_once __DIR__ . '/../Views/home.php';
10     }
11 }
```

4.2. Intégration de l'interface : Création de la Vue

Crée le fichier **src/Views/home.php**. Intègre la structure HTML5. Applique la méthodologie CSS **BEM (Block, Element, Modifier)** (tu peux te renseigner en ligne sur ce [lien](#)) imposée pour l'intégration de la maquette.

```
1  <!DOCTYPE html>
2  <html lang="fr">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Dashboard - Gestion des utilisateurs</title>
8  </head>
9
10 <body class="dashboard">
11     <header class="dashboard_header">
12         <h1 class="dashboard_title dashboard_title--main">Liste des Utilisateurs</h1>
13     </header>
14     <main class="dashboard_content">
15         <!-- Les données seront injectées ici -->
16     </main>
17 </body>
18 </html>
```


4.3. Gestion des données : Création du Modèle

Crée le fichier **src/Models/User.php**. Ce modèle encapsule la logique métier et les interactions avec la table **users**. Il exploite le **Singleton Database** pour récupérer l'instance **PDO**.

```
1  <?php
2
3  namespace App\Models;
4
5  use App\Config\Database;
6  use PDO;
7
8  class User
9  {
10     private PDO $db;
11
12     public function __construct()
13     {
14         $this->db = Database::getInstance()->getConnection();
15     }
16
17     public function getAll(): array
18     {
19         $query = "SELECT users.id, users.nom, users.email, roles.nom as role_nom
20                     FROM users
21                     JOIN roles ON users.roleId = roles.id";
22         $stmt = $this->db->query($query);
23
24         return $stmt->fetchAll();
25     }
26 }
```

4.4. Finalisation du cycle MVC : Transmission des données

Modifie le fichier **HomeController.php** pour instancier le modèle **User**, récupérer le tableau d'utilisateurs et rendre cette variable accessible dans la vue.

```

1 public function index(): void
2 {
3     $userModel = new User();
4     $users = $userModel->getAll(); // La variable $users est maintenant disponible dans la vue
5
6     require_once __DIR__ . '/../Views/home.php';
7 }

```

Dans le fichier **src/Views/home.php**, utilise une boucle **foreach** pour itérer sur le tableau **\$users** et afficher les données dans la structure HTML.

```

<!DOCTYPE html>
<html lang="fr">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Dashboard - Gestion des utilisateurs</title>
</head>

<body class="dashboard">
    <header class="dashboard_header">
        <h1 class="dashboard_title dashboard_title--main">Liste des Utilisateurs</h1>
        <a href="/user/create" class="dashboard_button dashboard_button--primary">Ajouter un utilisateur</a>
    </header>

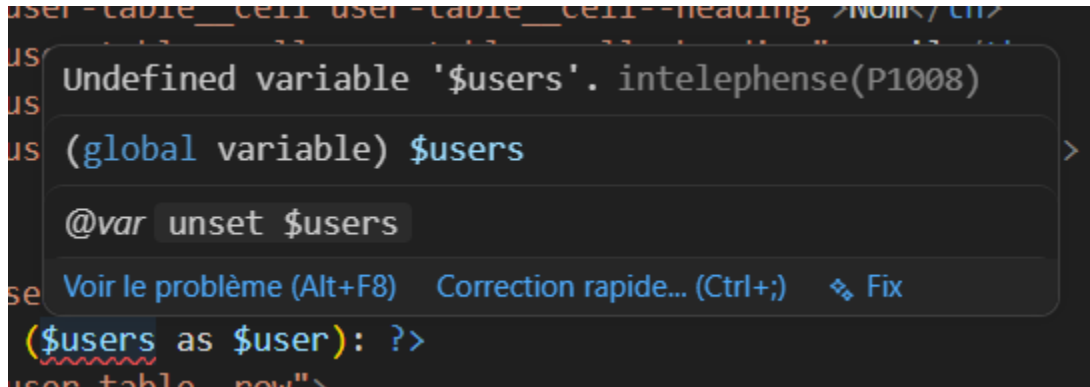
    <main class="dashboard_content">
        <table class="user-table">
            <thead class="user-table_head">
                <tr class="user-table_row user-table_row--header">
                    <th class="user-table_cell user-table_cell--heading">ID</th>
                    <th class="user-table_cell user-table_cell--heading">Nom</th>
                    <th class="user-table_cell user-table_cell--heading">Email</th>
                    <th class="user-table_cell user-table_cell--heading">Rôle</th>
                    <th class="user-table_cell user-table_cell--heading">Actions</th>
                </tr>
            </thead>
            <tbody class="user-table_body">
                <?php foreach ($users as $user): ?>
                    <tr class="user-table_row">
                        <td class="user-table_cell"><?= htmlspecialchars($user['id']) ?></td>
                        <td class="user-table_cell"><?= htmlspecialchars($user['nom']) ?></td>
                        <td class="user-table_cell"><?= htmlspecialchars($user['email']) ?></td>
                        <td class="user-table_cell">
                            <span class="role-badge role-badge--<?= strtolower(htmlspecialchars($user['role_nom'])) ?>">
                                <?= htmlspecialchars($user['role_nom']) ?>
                            </span>
                        </td>
                        <td class="user-table_cell user-table_cell--actions">
                            <a href="/user/edit?id=<?= $user['id'] ?>" class="button button--edit">Modifier</a>
                            <a href="/user/delete?id=<?= $user['id'] ?>" class="button button--delete">Supprimer</a>
                        </td>
                    </tr>
                <?php endforeach; ?>
            </tbody>
        </table>
    </main>
</body>
</html>

```

Voici mon code HTML. Bien sûr tu peux avoir le tien car il existe plusieurs manières de faire pour obtenir le même résultat. Par contre, je voudrais que nous nous concentrons sur 2 choses sur l'image :

1. Gestion des erreurs

La première chose nous allons régler le problème de `$users` qui est souligné en rouge :



Au survole on peut avoir des détails de l'erreur. En général on peut donc voir une partie des erreurs et pouvoir les corriger **avant de mettre en production**. Ce qui permet de coder **directement avec le moins d'erreurs** possible. Cela contribue fortement à l'**expérience développeur** (DX: Developer Experience).

Ici on remarque que c'est **intelephense** qui soulève l'erreur. Voici une petite explication :

1. Analyse Statique et Résolution des Faux Positifs (Intelephense)

Intelephense et son rôle

Intelephense est un serveur de langage PHP, massivement utilisé comme extension dans les éditeurs de code (comme VS Code). Il effectue une analyse statique du code en temps réel. Ses fonctions principales sont l'autocomplétion (IntelliSense), le formatage, la définition des types et la détection d'erreurs avant exécution (syntaxe, variables non définies, appels de méthodes inexistantes).

Origine de l'avertissement Undefined variable '\$users'

Ce message dans la vue home.php est un faux positif. L'analyseur statique d'Intelephense évalue le fichier home.php de manière isolée. Il ne trace pas le flux d'exécution de l'application et ignore que la variable **\$users** est générée par le **HomeController.php** juste avant l'appel `require_once`. Le code PHP est valide et s'exécute correctement sur le serveur, mais l'éditeur signale un manque de contexte local.

Résolution par typage PHPDoc

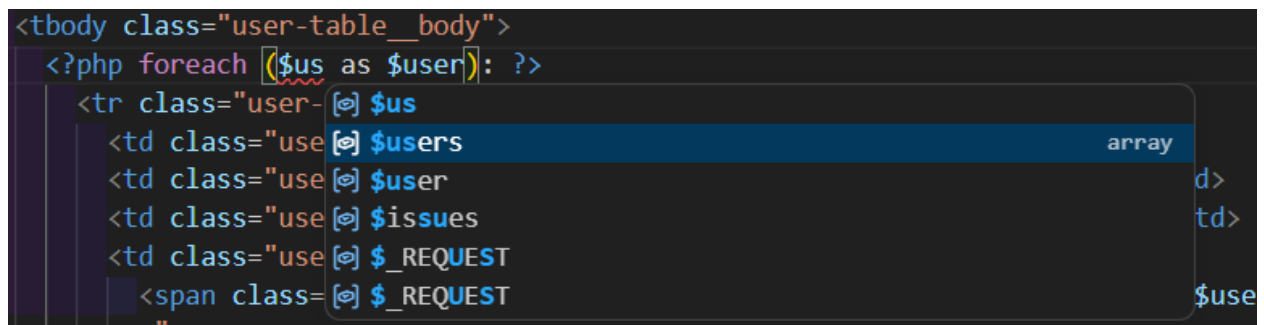
La solution exige de déclarer explicitement la variable dans la vue pour informer l'analyseur statique. Il faut utiliser un commentaire formaté PHPDoc au tout début du fichier HTML/PHP.



```

1  <?php
2  /** @var array $users */
3  ?>
4  <!-- Suite du code HTML -->
  
```

Cette instruction (`@var`) indique à Intelephense le type (array) et le nom (`$users`) de la variable injectée. Cela supprime l'avertissement visuel dans l'éditeur et active l'autocomplétion pour le contenu de cette variable dans le reste du fichier.



```

<tbody class="user-table_body">
  <?php foreach ($us as $user): ?>
    <tr class="user- [?] $us
      <td class="use [?] $users array
      <td class="use [?] $user d>
      <td class="use [?] $issues td>
      <td class="use [?] $_REQUEST
      <span class= [?] $_REQUEST $use
    </tr>
  </foreach>
</tbody>
  
```

Vois-tu l'intérêt d'avoir les bons outils pour développer et d'être rigoureux dans la documentation et le respect des bonnes pratiques ? Cela facilite toi et les futures personnes qui vont lire ton code.

2. Standards de l'Agence : Conventions de nommage et Méthodologie BEM

L'agence impose des conventions strictes pour garantir l'uniformité, la lisibilité et la maintenabilité du code par l'ensemble de l'équipe technique.

1. Conventions de nommage PHP (Standards PSR)

Le développement backend doit respecter les standards internationaux PHP-FIG (PSR-1 et PSR-12).

- **Classes et Espaces de noms (Namespaces)** : Utilisation stricte du PascalCase. Chaque mot commence par une majuscule.
 - **Exemples** : HomeController, Database, User.
- **Méthodes et Propriétés** : Utilisation du camelCase. Le premier mot est en minuscules, l'initiale des mots suivants est en majuscule.
 - **Exemples** : getAll(), getInstance(), \$userModel.
- **Variables générales** : Utilisation du camelCase.
- **Données SQL** : Les variables représentant directement des champs de la base de données conservent le format snake_case (mots séparés par des tirets du bas) pour correspondre au schéma de modélisation.
 - **Exemple** : \$user['role_id']. *(pour ma part j'ai utilisé camelCase car je préfère)*

2. Méthodologie CSS BEM (Block, Element, Modifier)

Mathilde conçoit les maquettes sous forme de composants isolables. L'intégration CSS doit refléter cette architecture modulaire via la convention BEM. L'objectif est de supprimer les sélecteurs CSS imbriqués (qui alourdissent la spécificité) et d'éviter les conflits de style.

Block (Bloc) : Entité conceptuellement indépendante. C'est le conteneur racine du composant.

- Syntaxe : Nom court en minuscules, mots séparés par un tiret simple.
 - Exemples : **.dashboard**, **.user-table**, **.button**.

Element (Élément) : Partie fonctionnelle ou structurelle liée à un bloc. Un élément n'a aucune signification stylistique en dehors de son bloc parent.

- Syntaxe : Nom du bloc parent suivi de deux underscores __ puis nom de l'élément.
 - Exemples : **.dashboard__header**, **.user-table__row**, **.user-table__cell**.

Modifier (Modificateur) : Variante stylistique appliquée à un bloc ou à un élément pour en altérer l'apparence (taille, couleur) ou l'état (actif, inactif).

- Syntaxe : Nom de la cible suivi de deux tirets -- puis valeur du modificateur.
 - Exemples : **.dashboard__title--main**, **.button--delete**, **.button--edit**.

Analyse le code de la vue home.php. Le bouton d'action utilise l'association de classes **class="button button--delete"**. La classe **.button** (le bloc) applique le socle commun (marges, typographie, structure), tandis que la classe **.button--delete** (le modificateur) surcharge uniquement la couleur d'arrière-plan en rouge. Cette approche est obligatoire pour valider la revue de code front-end.

Prends le temps de bien assimiler ces concepts car si certaines choses sont nouvelles pour toi; nous pensons qu'il faut que tu maîtrises ses bases. Si c'est tout bon, nous allons simplement faire le css et voir comment l'importer. Tu vas voir c'est tout bête.

4.5. Création et Importation de la Feuille de Style BEM

Crée l'arborescence **public/assets/css/** et crée le fichier **style.css**. Ce fichier contient l'implémentation stricte des classes BEM définies dans la vue.

Voici mon cas :

```

1  * {
2      box-sizing: border-box;
3      margin: 0;
4      padding: 0;
5  }
6
7  body {
8      font-family: Arial, sans-serif;
9      background-color: #f4f6f9;
10     color: #333;
11 }
12
13 /* Bloc : Dashboard */
14 .dashboard {
15     padding: 2rem;
16     max-width: 1200px;
17     margin: 0 auto;
18 }
19
20 .dashboard__header {
21     display: flex;
22     justify-content: space-between;
23     align-items: center;
24     margin-bottom: 2rem;
25 }
26
27 .dashboard__title--main {
28     font-size: 1.75rem;
29     color: #2c3e50;
30 }

```

```

1  /* Bloc / Éléments : Boutons */
2  .button, .dashboard__button {
3      padding: 0.5rem 1rem;
4      text-decoration: none;
5      border-radius: 4px;
6      font-weight: 600;
7      color: #fff;
8      cursor: pointer;
9  }
10
11 .dashboard__button--primary {
12     background-color: #3498db;
13 }
14
15 .button--edit {
16     background-color: #f39c12;
17 }
18
19 .button--delete {
20     background-color: #e74c3c;
21 }

```

```

1  /* Bloc : User Table */
2  .user-table {
3      width: 100%;
4      border-collapse: collapse;
5      background: #fff;
6      border-radius: 8px;
7      overflow: hidden;
8      box-shadow: 0 4px 6px rgba(0,0,0,0.05);
9  }
10
11 .user-table__cell {
12     padding: 1rem;
13     text-align: left;
14     border-bottom: 1px solid #eee;
15 }
16
17 .user-table__row--header .user-table__cell {
18     background-color: #2c3e50;
19     color: #fff;
20     text-transform: uppercase;
21     font-size: 0.85rem;
22 }
23
24 .user-table__row:hover {
25     background-color: #f9f9f9;
26 }
27
28 .user-table__cell--actions {
29     display: flex;
30     gap: 0.5rem;
31 }

```

```

1  /* Bloc : Role Badge */
2  .role-badge {
3      padding: 0.25rem 0.75rem;
4      border-radius: 12px;
5      font-size: 0.75rem;
6      font-weight: bold;
7      color: #fff;
8      text-transform: uppercase;
9  }
10
11 .role-badge--administrateur {
12     background-color: #8e44ad;
13 }
14
15 .role-badge--utilisateur {
16     background-color: #7f8c8d;
17 }

```

1. Importation dans la Vue

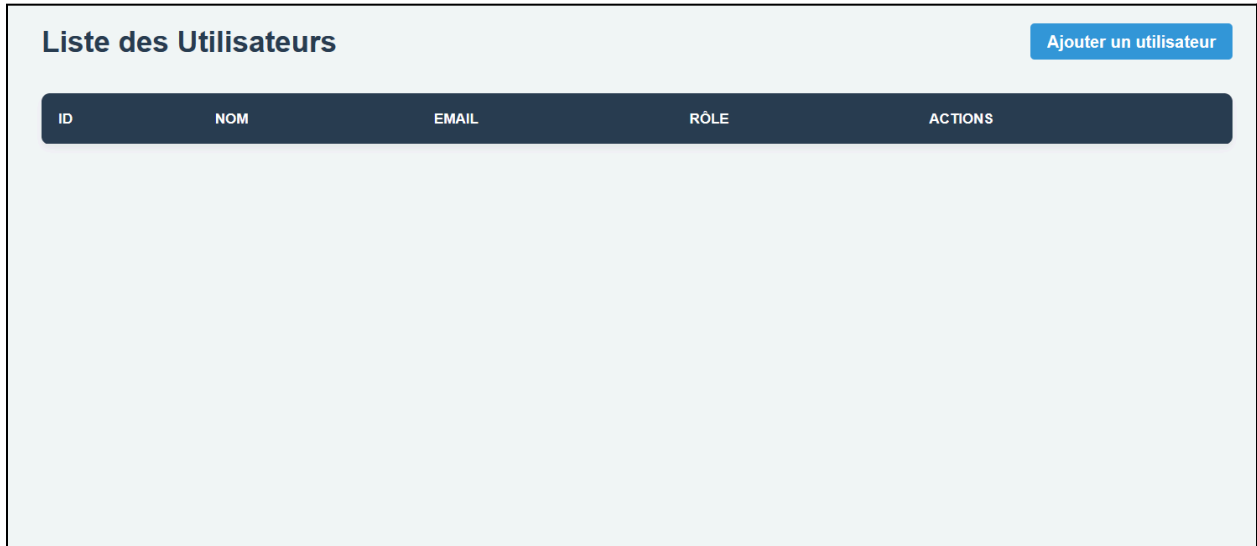
Ouvre le fichier **src/Views/home.php**. Ajoute la balise **<link>** dans l'en-tête pour charger le fichier CSS. Le chemin doit être absolu par rapport au dossier public configuré comme racine du serveur.

```

1  <link rel="stylesheet" href="/assets/css/style.css">

```


J'obtiens donc cette interface :



ID	NOM	EMAIL	RÔLE	ACTIONS
----	-----	-------	------	---------

5. Phase de Développement Autonome : Authentification, CRUD et Sécurité

Le tableau de bord (/) est fonctionnel mais la base de données est vide. La prochaine étape consiste à implémenter la logique métier complète en autonomie. Voici le cahier des charges technique des fonctionnalités à développer.

5.1. Autonomie et Résolution de Problèmes

- Le développement logiciel implique des blocages fréquents. La capacité à trouver des solutions indépendamment est exigée.
- Consultation obligatoire de la documentation officielle (PHP.net, MDN Web Docs, documentation AltoRouter) avant toute demande d'assistance.
- En cas d'erreur fatale ou de comportement inattendu, analyser la stack trace, lire les logs Apache/PHP de Laragon et rechercher le message d'erreur exact en ligne (StackOverflow, forums spécialisés). La résolution autonome de bugs fait partie intégrante de l'évaluation technique.

5.2. Authentification et Sessions Natives

- **Inscription (Register)** : Crée la vue, le contrôleur et la méthode de modèle pour l'ajout initial d'un utilisateur. Le mot de passe doit obligatoirement être haché en base de données avec la fonction native `password_hash()`.
- **Connexion (Login)** : Développe le formulaire et la validation des identifiants avec `password_verify()`.
- **Gestion des Sessions** : Implémente le système d'authentification via les sessions natives PHP (`session_start()`). Stocke l'identifiant et le rôle de l'utilisateur dans la superglobale `$_SESSION`. Sécurise le cookie de session en forçant les paramètres `HttpOnly`, `Secure` et `SameSite=Strict`. Développe la logique de déconnexion avec `session_destroy()`.

5.3. Finalisation du CRUD Utilisateurs

Développer les opérations manquantes dans l'architecture MVC :

- **Create** : Route, contrôleur et vue pour l'ajout d'utilisateurs depuis le tableau de bord.
- **Update** : Formulaire d'édition pour modifier le nom, l'email ou le rôle d'un compte existant.
- **Delete** : Logique de suppression d'une entrée dans la table `users`.

5.4. Sécurité et Contrôle des Rôles

- **Prévention des Injections SQL** : Interdiction stricte d'utiliser des variables directement dans les chaînes SQL (`query()`). Utiliser systématiquement les requêtes préparées avec `$db->prepare()`, `bindValue()` et `execute()`.
- **Prévention XSS** : Maintenir l'utilisation de `htmlspecialchars()` pour chaque donnée dynamique affichée dans les vues.
- **Gestion des Autorisations (ACL)** : Restreindre l'accès aux routes du CRUD. Extraire le rôle de l'utilisateur à partir du JWT. Seul le profil "Administrateur" est autorisé à créer, modifier ou supprimer des membres. Les utilisateurs standards tentant d'accéder à ces routes doivent être bloqués (Code HTTP 403 Forbidden).

5.5. Rappel des Standards

- Maintien de l'architecture MVC et du typage strict PHP.
- Application systématique de la méthodologie CSS BEM pour les nouvelles interfaces (formulaire d'inscription/connexion).
- Validation des données entrantes côté client (JS Vanilla) et côté serveur (PHP).

5.5. Livrable : Documentation Technique

- Rédiger un fichier **README.md** exhaustif à la racine du projet.
- Contenu exigé : processus de déploiement local (Composer, base de données), configuration des variables d'environnement (.env), description des routes disponibles, et explication technique de l'implémentation du système JWT choisi.